Lesson 18 Structured Note: Postfix Evaluation Theory and Procedure

0. Current Position in the Course

You are now starting **Lesson 18** of the stack curriculum.

Before Lesson 18, you completed these major parts:

Phase	Lessons	Main Focus
Phase 1	Lessons 1–4	Stack basics and array-stack implementation
Phase 2	Lessons 5–8	Linked-list stack implementation in C and C++
Phase 3	Lessons 9–12	Parenthesis and bracket matching using stacks
Phase 4	Lessons 13–17	Infix, prefix, postfix, precedence, associativity, and infix-to-postfix conversion

Now you are entering:

Phase 5: Postfix Evaluation and Integration

In Lesson 17, you learned how to convert an expression from **infix** to **postfix**.

Example:

```
Infix:   3 + 5
Postfix: 35+
```

Lesson 18 answers the next question:

> After converting an expression into postfix form, how do we calculate the final answer?

That process is called:

```
Postfix evaluation
```

---

1. Lesson 18 Goal

By the end of this lesson, you should understand:

- What postfix evaluation means.
- Why postfix expressions are easy to evaluate using a stack.
- Why the stack stores integers during postfix evaluation.
- How to scan a postfix expression from left to right.
- What to do when you see an operand.
- What to do when you see an operator.
- Why an operator needs two popped values.
- Why pop order matters.
- Why subtraction and division are sensitive to pop order.
- Why postfix evaluation does not need precedence rules.
- How to dry-run a postfix expression step by step.
- How to write the basic algorithm for postfix evaluation.
- How beginner C-style logic for postfix evaluation works.
- What common beginner mistakes to avoid.

---

2. Quick Review: Infix and Postfix

An expression can be written in different forms.

Infix Notation

In **infix notation**, the operator is written between the operands.

Example:

```
3 + 5
```

Here:

Symbol	Role
`3`	Operand
`+`	Operator
`5`	Operand

So the structure is:

```
Operand Operator Operand
```

Humans usually write expressions in infix form.

Postfix Notation

In **postfix notation**, the operator is written after the operands.

Example:

```
35+
```

This means the same thing as:

```
3 + 5
```

The structure is:

```
Operand Operand Operator
```

So:

```
35+
```

means:

```
3 + 5
```

---

3. What Is Postfix Evaluation?

Postfix evaluation means calculating the final answer of a postfix expression.

Example:

```
35+
```

The final answer is:

```
8
```

Why?

Because:

```
3 + 5 = 8
```

A postfix expression is evaluated by scanning it from left to right and using a stack.

---

4. Why Do We Use a Stack for Postfix Evaluation?

A stack is useful because postfix notation gives operands first and operators later.

Example:

```
35+
```

When we scan from left to right:

1. We see `3`.
2. We do not know the operator yet, so we save it.
3. We see `5`.
4. We still need to save it.
5. We see `+`.
6. Now we know we must add the two most recent numbers.

A stack is perfect for this because it follows:

```
LIFO = Last In, First Out
```

The most recent operands are needed first when an operator appears.

---

5. Main Rule of Postfix Evaluation

When scanning a postfix expression:

```
If the symbol is an operand, push it onto the stack.
If the symbol is an operator, pop two values, calculate, and push the result.
```

Simple table:

Symbol Type	Action
Operand	Push it onto the stack
Operator	Pop two values, apply the operator, push the result

---

6. Simple Example: Evaluating `35+`

Expression:

```
35+
```

Initial stack:

```
empty
```

Step-by-step trace:

Step	Symbol	Action	Stack After Action
1	`3`	Push 3	`3`
2	`5`	Push 5	`3, 5`
3	`+`	Pop 5 and 3, calculate `3 + 5 = 8`, push 8	`8`

Final stack:

```
8
```

Final answer:

```
8
```

---

7. Why the Stack Stores Integers Now

In earlier lessons, such as infix-to-postfix conversion, the stack stored operators like:

```
+ - * /
```

Those were characters.

But in postfix evaluation, we are doing arithmetic.

So the stack must store numbers such as:

```
3, 5, 15, 18
```

That means Lesson 18 uses an:

```
Integer stack
```

not a character stack.

Important idea:

```
In infix-to-postfix conversion, the stack stores operators.
In postfix evaluation, the stack stores numbers.
```

---

8. Operand Rule

An **operand** is a number used in the expression.

Examples:

```
0 1 2 3 4 5 6 7 8 9
```

In this beginner lesson, we mainly work with single-digit operands.

When we see an operand:

```
Push it onto the stack.
```

Example:

Expression:

```
82/
```

When we see `8`:

```
push 8
```

When we see `2`:

```
push 2
```

The stack becomes:

```
8, 2
```

---

9. Operator Rule

An **operator** performs an operation.

Common operators are:

Operator	Meaning
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division

When we see an operator:

1. Pop the first value.
2. Pop the second value.

3. Apply the operator correctly.
4. Push the result back onto the stack.

General form:

```
x2 = pop()
x1 = pop()
result = x1 operator x2
push(result)
```

---

10. Very Important Rule: Pop Order Matters

This is one of the most important ideas in Lesson 18.

When an operator appears, we pop two values.

The **first popped value** is the **right operand**.

The **second popped value** is the **left operand**.

So the correct pattern is:

```
x2 = pop()   // right operand
x1 = pop()   // left operand
result = x1 operator x2
```

Do not reverse them.

---

11. Why Pop Order Matters for Subtraction

Suppose the stack contains:

```
3, 5
```

The top is:

```
5
```

Now suppose we see the operator:

```
-
```

We pop two values:

```
first pop  -> 5
second pop -> 3
```

The correct expression is:

```
3 - 5
```

not:

```
5 - 3
```

So:

```
x2 = 5
x1 = 3
result = x1 - x2
result = 3 - 5
result = -2
```

Important:

```
The first popped value goes on the right side.
The second popped value goes on the left side.
```

---

12. Why Pop Order Matters for Division

Division is also order-sensitive.

Expression:

```
82/
```

This means:

```
8 / 2
```

Dry run:

Step	Symbol	Action	Stack After Action
1	`8`	Push 8	`8`
2	`2`	Push 2	`8, 2`
3	`/`	Pop 2, pop 8, calculate `8 / 2 = 4`, push 4	`4`

Correct answer:

```
4
```

The correct calculation is:

```
x2 = pop() = 2
x1 = pop() = 8
result = x1 / x2
result = 8 / 2
result = 4
```

Wrong calculation:

```
2 / 8
```

That is not what the postfix expression means.

---

13. Operators That Are Sensitive to Order

Addition and multiplication are less sensitive because:

```
3 + 5 = 5 + 3
3 * 5 = 5 * 3
```

But subtraction and division are very sensitive:

```
3 - 5 is not the same as 5 - 3
8 / 2 is not the same as 2 / 8
```

So always use:

```
x2 = pop()
x1 = pop()
result = x1 operator x2
```

---

14. Full Lesson 18 Dry Run

Evaluate this postfix expression:

```
35*62/+4-
```

We scan from left to right.

Initial stack:

```
empty
```

Step-by-Step Trace

Step	Symbol	Action	Stack After Action
1	`3`	Push 3	`3`
2	`5`	Push 5	`3, 5`
3	`*`	Pop 5, pop 3, calculate `3 * 5 = 15`, push 15	`15`
4	`6`	Push 6	`15, 6`
5	`2`	Push 2	`15, 6, 2`
6	`/`	Pop 2, pop 6, calculate `6 / 2 = 3`, push 3	`15, 3`
7	`+`	Pop 3, pop 15, calculate `15 + 3 = 18`, push 18	`18`
8	`4`	Push 4	`18, 4`
9	`-`	Pop 4, pop 18, calculate `18 - 4 = 14`, push 14	`14`

Final stack:

```
14
```

Final answer:

```
14
```

---

15. Understanding the Full Dry Run in Parts

Expression:

```
35*62/+4-
```

This expression is evaluated in this order:

Part 1: `35*`

```
3 * 5 = 15
```

Stack becomes:

```
15
```

Part 2: `62/`

```
6 / 2 = 3
```

Stack becomes:

```
15, 3
```

Part 3: `+`

```
15 + 3 = 18
```

Stack becomes:

```
18
```

Part 4: `4-`

```
18 - 4 = 14
```

Final answer:

```
14
```

---

16. Why Postfix Evaluation Does Not Need Precedence Rules

In infix notation, precedence matters.

Example:

```
3 + 5 * 2
```

In infix form, we must know that multiplication happens before addition.

So:

```
3 + 5 * 2
```

means:

```
3 + (5 * 2)
```

But postfix notation already stores the correct order of operations.

The postfix form is:

```
352*+
```

When we evaluate postfix, we do not ask:

```
Which operator has higher precedence?
```

Instead, we simply scan left to right and execute operators when they appear.

---

17. Example Showing No Precedence Needed

Evaluate:

```
352*+
```

Dry run:

Step	Symbol	Action	Stack After Action
1	`3`	Push 3	`3`
2	`5`	Push 5	`3, 5`
3	`2`	Push 2	`3, 5, 2`
4	`*`	Pop 2, pop 5, calculate `5 * 2 = 10`, push 10	`3, 10`
5	`+`	Pop 10, pop 3, calculate `3 + 10 = 13`, push 13	`13`

Final answer:

```
13
```

Notice:

We did not check whether `*` has higher precedence than `+`.

The postfix expression already placed `*` before `+` in the execution order.

---

18. Precedence vs Postfix Execution

This is an important difference.

Stage	What Happens?
Infix-to-postfix conversion	Precedence is used to decide the correct postfix order

Stage	What Happens?
Postfix evaluation	Precedence is not needed because the order is already fixed

Simple idea:

```
Precedence is needed before postfix is created.
Precedence is not needed after postfix is created.
```

---

19. Simple Algorithm for Postfix Evaluation

Algorithm:

```
evaluatePostfix(postfix):
    create an empty integer stack

    scan postfix from left to right:
        if current symbol is an operand:
            convert it to integer
            push it

        else if current symbol is an operator:
            x2 = pop()
            x1 = pop()

            result = x1 operator x2

            push result

    final answer = pop()
    return final answer
```

---

20. Algorithm in Very Simple Words

1. Start with an empty stack.
2. Read the postfix expression from left to right.
3. If you see a number, push it.
4. If you see an operator, pop two numbers.
5. Use the second popped number as the left operand.
6. Use the first popped number as the right operand.
7. Calculate the result.

8. Push the result back onto the stack.
9. At the end, the stack contains the final answer.

---

21. Beginner C-Style Logic

This is the basic idea of a postfix evaluation function:

```c
int evaluatePostfix(char *postfix) {
    int i;
    int x1, x2, r;

    for (i = 0; postfix[i] != '\0'; i++) {
        if (isOperand(postfix[i])) {
            push(postfix[i] - '0');
        } else {
            x2 = pop();
            x1 = pop();

            switch (postfix[i]) {
                case '+':
                    r = x1 + x2;
                    break;
                case '-':
                    r = x1 - x2;
                    break;
                case '*':
                    r = x1 * x2;
                    break;
                case '/':
                    r = x1 / x2;
                    break;
            }

            push(r);
        }
    }

    return pop();
}
```

This code is a beginner-level version of the main logic.

---

22. Meaning of the Main Variables

In the code:

```
int i;
int x1, x2, r;
```

The meanings are:

Variable	Meaning
`i`	Scans the postfix expression from left to right
`x2`	First popped value, used as the right operand
`x1`	Second popped value, used as the left operand
`r`	Result after applying the operator

Example:

For:

```
82/
```

We get:

```
x2 = 2
x1 = 8
r = 8 / 2 = 4
```

---

23. Why We Use `postfix[i] - '0'`

The postfix expression is stored as a string.

Example:

```
char postfix[] = "35+";
```

Inside the string, `3` is stored as a character:

```
'3'
```

But for arithmetic, we need the integer:

```
3
```

So we convert a digit character into an integer using:

```
postfix[i] - '0'
```

Examples:

Character	Conversion	Integer
`'3'`	`'3' - '0'`	`3`
`'5'`	`'5' - '0'`	`5`
`'8'`	`'8' - '0'`	`8`

Important:

```
'3' is a character.
3 is an integer.
```

For arithmetic, we need the integer.

---

24. What `isOperand()` Means Here

In postfix evaluation, `isOperand()` checks whether the current character is a number.

For this beginner version:

```
Operands are digits.
Operators are +, -, *, and /.
```

So:

Symbol	Operand?
`3`	Yes
`8`	Yes
`+`	No
`*`	No

A simple idea is:

```
If it is not an operator, treat it as an operand.
```

But in stronger programs, we should check carefully that it is actually a digit.

---

25. What the `switch` Does

The `switch` checks which operator appeared.

Example:

```c
switch (postfix[i]) {
    case '+':
        r = x1 + x2;
        break;
    case '-':
        r = x1 - x2;
        break;
    case '*':
        r = x1 * x2;
        break;
    case '/':
        r = x1 / x2;
        break;
}
```

Meaning:

Operator	Calculation
`+`	`x1 + x2`
`-`	`x1 - x2`

Operator	Calculation
`*`	`x1 * x2`
`/`	`x1 / x2`

After calculating, the result is pushed back onto the stack:

```
push(r);
```

---

26. Why the Result Is Pushed Back

After an operator is evaluated, the result may be needed by a later operator.

Example:

```
23+4*
```

First:

```
2 + 3 = 5
```

But the expression is not finished.

The result `5` must be used with `4` and `*`.

So we push `5` back onto the stack.

Then:

```
5 * 4 = 20
```

Final answer:

```
20
```

---

27. Dry Run: `23+4*`

Expression:

```
23+4*
```

Initial stack:

```
empty
```

Step-by-step trace:

Step	Symbol	Action	Stack After Action
1	`2`	Push 2	`2`
2	`3`	Push 3	`2, 3`
3	`+`	Pop 3, pop 2, calculate `2 + 3 = 5`, push 5	`5`
4	`4`	Push 4	`5, 4`
5	`*`	Pop 4, pop 5, calculate `5 * 4 = 20`, push 20	`20`

Final answer:

```
20
```

---

28. Dry Run: `82/3-`

Expression:

```
82/3-
```

Initial stack:

```
empty
```

Step-by-step trace:

Step	Symbol	Action	Stack After Action
1	`8`	Push 8	`8`
2	`2`	Push 2	`8, 2`
3	`/`	Pop 2, pop 8, calculate `8 / 2 = 4`, push 4	`4`
4	`3`	Push 3	`4, 3`
5	`-`	Pop 3, pop 4, calculate `4 - 3 = 1`, push 1	`1`

Final answer:

```
1
```

---

29. Common Beginner Mistake 1: Reversing Pop Order

Wrong:

```
x1 = pop();
x2 = pop();
r = x1 - x2;
```

Why is this wrong?

Because the first popped value should be the right operand, not the left operand.

Correct:

```
x2 = pop();
x1 = pop();
r = x1 - x2;
```

Always remember:

```
First pop = right side
Second pop = left side
```

---

30. Common Beginner Mistake 2: Pushing Characters Instead of Integers

Wrong idea:

```
push(postfix[i]);
```

If `postfix[i]` is `'3'`, this pushes the character `'3'`, not the integer `3`.

Correct idea:

```
push(postfix[i] - '0');
```

This converts the character digit into an integer digit.

---

31. Common Beginner Mistake 3: Using Precedence During Evaluation

Some beginners try to compare operators during postfix evaluation.

That is not needed.

Precedence is already handled before evaluation when the infix expression is converted to postfix.

During postfix evaluation, just scan left to right.

Correct idea:

```
Operand -> push
Operator -> pop two values, calculate, push result
```

No precedence comparison is needed.

---

32. Common Beginner Mistake 4: Forgetting to Push the Result

When an operator is processed, the result must be pushed back.

Wrong idea:

```
pop two values
calculate result
move on
```

Correct idea:

```
pop two values
calculate result
push result
```

Why?

Because the result may be needed by the next operator.

---

33. Common Beginner Mistake 5: Popping When There Are Not Enough Values

Every binary operator needs two operands.

So when we see an operator, the stack should have at least two values.

Example of invalid postfix:

```
3+
```

There is only one operand before `+`.

The operator `+` needs two operands.

So this expression is invalid.

For beginner lessons, we often assume the postfix expression is valid.

---

34. Limitations of the Beginner Version

The beginner version usually handles:

```
Single-digit operands
Operators: +, -, *, /
```

```
Valid postfix expressions
Integer arithmetic
```

It does not fully handle:

```
Multi-digit numbers like 12 or 345
Spaces between tokens
Negative numbers
Floating-point numbers
Invalid postfix expressions
Division by zero checks
Advanced operators
```

These can be added later after the basic idea is clear.

---

35. Complete Concept Summary

Postfix evaluation uses a stack.

The rules are:

```
Operand -> push onto stack
Operator -> pop two values, calculate, push result
```

The most important pop rule is:

```
x2 = pop()   // right operand
x1 = pop()   // left operand
result = x1 operator x2
```

At the end:

```
The final value left in the stack is the answer.
```

---

36. Lesson 18 Key Sentence

You understand Lesson 18 if you can explain this sentence:

In postfix evaluation, operands are pushed, operators pop two values, the second popped value is the left operand, the first popped value is the right operand, and the result is pushed back.

---

37. Beginner Checklist

Before moving to Lesson 19, make sure you can do these:

- I can explain what postfix evaluation means.
- I can scan a postfix expression from left to right.
- I can push operands onto the stack.
- I can pop two values when I see an operator.
- I know that the first pop is the right operand.
- I know that the second pop is the left operand.
- I can correctly evaluate subtraction and division.
- I know why postfix evaluation does not need precedence.
- I can dry-run `35*62/+4-`.
- I understand why the stack stores integers.
- I understand why digit characters need conversion using `- '0'`.
- I can explain why the result must be pushed back.

---

38. Practice Tasks

Practice 1

Evaluate:

```
23+4*
```

Expected result:

```
20
```

Practice 2

Evaluate:

```
82/3-
```

Expected result:

```
1
```

Practice 3

Evaluate:

```
52-
```

Think carefully about pop order.

Expected result:

```
3
```

Because:

```
5 - 2 = 3
```

Practice 4

Evaluate:

```
93/
```

Expected result:

```
3
```

Because:

```
9 / 3 = 3
```

Practice 5

Evaluate:

```
234*+
```

Step idea:

```
3 * 4 = 12
2 + 12 = 14
```

Expected result:

```
14
```

---

39. Final Lesson 18 Revision Table

Concept	Simple Meaning
Postfix evaluation	Calculating the answer of a postfix expression
Operand	A number, such as `3` or `8`
Operator	A symbol, such as `+`, `-`, `*`, `/`
Operand action	Push onto stack
Operator action	Pop two values, calculate, push result
First pop	Right operand
Second pop	Left operand
Stack type	Integer stack
Precedence needed?	No
Final answer	Last value left in the stack

---

40. Final Takeaway

Postfix evaluation is simple once you remember the stack rule.

Read the expression from left to right.

Numbers go into the stack.

Operators take two numbers out of the stack.

The result goes back into the stack.

At the end, the stack contains the final answer.

The most important rule is:

```
x2 = pop()
x1 = pop()
result = x1 operator x2
```

This rule protects you from mistakes in subtraction and division.